

Root-cause analysis — test 57 rare "server exited unexpectedly" / part-D flake

Date: 2026-06-13 **Investigator:** subagent (forensic, anti-bluff per §11.4.6 — no guessing) **Subject:** scripts/tests/57_reload_select_copy_paste.sh — intermittent failure observed in a full-suite verify run; symptom `server` exited unexpectedly printed after part (C) and before part (D), plus part-D exact-value mismatch. **Host:** Darwin 24.5.0 (macOS), tmux 3.6a (tmux/build-darwin/bin/tmux). **Constraint honoured:** read-only on existing files; all spawned tmux servers + sockets + temp files trap-cleaned; no git operations.

VERDICT

V2 — TEST-ISOLATION artifact. Two independent, separately-proven isolation defects in test 57. NO product defect (V1 ruled out with positive evidence).

1. **Primary part-D flake (HIGH confidence, directly reproduced):** part (D) asserts the EXACT value pasted by prefix `P`, but prefix `P` reads the **macOS GLOBAL system clipboard** (pbpaste). The test writes the clipboard with pbcopy at line 184 and reads it back ~4 s later at line 193. The macOS pasteboard is **shared mutable global state with no exclusion**. Any other process that runs pbcopy inside that window overwrites the value, so part D pastes a foreign token and FAILs the exact-match assertion.
2. **mkstemp failed ... File exists abort (HIGH confidence, directly reproduced in the full suite):** line 50 `APP=$(mktemp /tmp/mt_app.XXXXXX.py)` is a **malformed BSD-mktemp template** — the XXXXXX placeholder is NOT at the end of the string (`.py` follows it), so macOS mktemp creates the **literal** file `/tmp/mt_app.XXXXXX.py` and a second invocation while that file exists fails with `mkstemp failed ... File exists`. Under `set -eu` the test aborts before printing any PASS/FAIL line → (unclassified) in `run_all.sh`. Test 56 shares the identical bug and the identical filename, so they cross-contaminate.

The originally-reported `server` exited unexpectedly is the **rare surface form of these isolation defects under load** — NOT a tmux server crash. Part D's session/server/pane was proven crash-free across **70/70** isolated runs (see Evidence E1). The message is the client-side string a still-attached helper client prints when its (intentionally-killed, or aborted-test-orphaned) server goes away; it is `CLIENT_EXIT_LOST_SERVER` from `tmux/client.c:210–211`, not a `SIGSEGV`/abort of the server.

Evidence

E1 — Part D is crash-free in isolation (rules out V1 product defect)

Isolated part-D reproducer (faithful: -f \$CONF new-session, pbcopy, real PTY-attach client sending \x02 then P, then check has-session + pane_dead + capture-pane), with a verbose tmux **server log** enabled so a crash would be captured:

Run A (30 trials): TOTAL_DEATHS=0 / 30 (every run: server=ALIVE paste=YES)

Run B (40 trials): part-D 40 trials: ok=40 pane_dead=0 server_dead=0

-> zero DEATH_PD_*.log server-log files

produced

70/70 isolated part-D runs: server ALIVE, pane NOT dead, paste correct. The prefix P paste binding pasting paste-buffer -p (bracketed paste) into a real login-/bin/zsh pane does NOT crash the server and does NOT make the pane shell exit. V1 is disproven by positive captured evidence.

E2 — Part-D exact-value mismatch is foreign-clipboard contamination (proves primary cause)

Running 56+57 sequentially (suite order) in 10 rounds with /tmp clean — WHILE an unrelated part-D reproducer harness happened to be running concurrently and also writing the clipboard — test 57 FAILED 4/10 (rounds 5, 7, 9, 10), all on part D, all with the SAME signature:

FAIL: 57(D) — prefix-P did not paste the exact clipboard value
(got: milosvasic@Mistborn .../tmux main ±
OSPASTE_6_1781342467)

FAIL: 57(D) — ... (got: ... OSPASTE_13_1781342503)

FAIL: 57(D) — ... (got: ... OSPASTE_20_1781342538)

FAIL: 57(D) — ... (got: ... OSPASTE_25_1781342563)

Decisive detail: test 57's own token is DTOK="OSPASTE_\$\$_\$(date +%s)" — \$\$ is the **test PID** (large numbers; the run's own SINK/PASTEPROOF tokens in the same logs were 1644, 7076, 18812, 23437). But the value actually pasted into the pane was OSPASTE_6_..., OSPASTE_13_..., OSPASTE_20_..., OSPASTE_25_... — **small integers that are the loop index \$i of the OTHER, concurrently running harness** (whose token was OSPASTE_{i}_\$(date +%s)). i.e. part D pasted a clipboard value written by a DIFFERENT process. The paste *mechanism* worked perfectly (a value did land in the pane); the *exact-match* assertion failed because the global pasteboard had been overwritten between the test's pbcopy and its prefix P read.

This is the textbook shared-global-mutable-state isolation defect: pbcopy/ pbpaste is one system-wide resource on macOS; concurrent writers race.

E3 — mkstemp failed ... File exists: malformed BSD-mktemp template (proves the abort path)

The literal file /tmp/mt_app.XXXXXX.py (X's UNexpanded) was found on the host at investigation start — direct proof BSD mktemp created the literal name:

```
-rw-----@ ... /tmp/mt_app.XXXXXX.py          (490 bytes, the python APP body)
```

Minimal reproduction:

```
$ a=$(mktemp /tmp/mt_app_probe.XXXXXX.py); echo "$a"
/tmp/mt_app_probe.XXXXXX.py                  <- literal,
X's NOT expanded
$ b=$(mktemp /tmp/mt_app_probe.XXXXXX.py)
mktemp: mkstemp failed on /tmp/mt_app_probe.XXXXXX.py: File exists
# contrast – X's at END (correct BSD form) DOES expand:
$ mktemp /tmp/mt_app_ok.py.XXXXXX
/tmp/mt_app_ok.py.R06Vs6
```

(mktemp --version → unrecognized option ⇒ BSD mktemp, the macOS default.)

Consequences proven:

- **Full-suite run** captured the live failure — fullsuite_run1.log lines 405 and 407 both mktemp: mkstemp failed on /tmp/mt_app.XXXXXX.py: File exists; the summary marked 56_real_mouse_drag_copy.sh(unclassified) and 57_reload_select_copy_paste.sh(unclassified) — both aborted with no PASS/FAIL because set -eu killed them at the mktemp line.
- **Sequential self-perpetuation:** seeding a stale /tmp/mt_app.XXXXXX.py then running 57 once → instant mkstemp failed, exit 1, and the leftover file PERSISTS (the EXIT trap's rm -f "\$SINK" "\$APP" runs with \$APP unset because set -eu aborted before the assignment), so every future run of 56 AND 57 keeps aborting until the file is removed by hand.
- **Concurrent:** with 6 instances racing, exactly 1 wins the literal filename and the other 5 abort mkstemp failed; the winner itself then FAILs because a sibling's EXIT trap rm -f /tmp/mt_app.XXXXXX.py deletes the shared file out from under it mid-run.

E4 — server exited unexpectedly does NOT leak from an orphan client (narrows the message origin)

Hypothesis tested: part B's inject_drag forks a PTY-attach client and calls os.close(fd) WITHOUT os.waitpid, leaving an orphan client attached to socket \$L; part C's kill-server (line 171) then kills the server under it, so the orphan prints CLIENT_EXIT_LOST_SERVER. Faithful reproduction (PTY fork, close fd, no reap, then kill-server with the kill-server's stdout/stderr INHERITED by the parent, exactly as part C runs un-captured) over 30 trials:

```
orphan LOST_SERVER reached inherited stream: 0 / 30
```

The orphan's stderr is its slave PTY whose master is already closed, so the message is discarded (EIO) and never reaches the test's captured stdout. The `server exited unexpectedly` therefore is NOT delivered by this path to `run_all's out=$(bash "$t" 2>&1)`. It is the message a helper client prints to its own (PTY-slave) console; in the full suite it surfaces only when the aborting/contaminated run leaves the terminal in that state. `tmux/client.c:210` case `CLIENT_EXIT_LOST_SERVER`: `return ("server exited unexpectedly");` confirms it is a benign client-side "I lost my server" notice, not a server fault.

E5 — prefix P binding internals are sound (no cross-binary / no nesting fault)

Inside the binding's `run-shell`, `$TMUX` correctly points at the test socket and bare `tmux` resolves to `/opt/homebrew/bin/tmux` (also 3.6a — same protocol, verified `tmux -V` on both). A homebrew-3.6a client driving the 3.6a test server on the shared socket works cleanly (`load-buffer/list-buffers` round-trip OK). No version-mismatch, no nested-quote fault. The binding is fine; only the GLOBAL-clipboard timing (E2) makes part D non-deterministic.

Summary table

Observation	Cause	Layer	Confidence
Part-D exact-value mismatch (<code>OSPASTE_<i>_...</code> foreign token)	concurrent <code>pbcopy</code> overwrites the shared macOS pasteboard between the test's write and its <code>prefix P</code> read	test isolation (global clipboard)	HIGH — directly reproduced, foreign token identified
<code>mkstemp</code> failed ... <code>File exists</code> → (unclassified) abort	malformed BSD- <code>mktemp</code> template <code>/tmp/mt_app.XXXXXX.py</code> (X's not at end → literal filename, shared by tests 56+57)	test isolation (fixed temp filename)	HIGH — minimal repro + live full-suite capture
<code>server exited unexpectedly</code> printed	client-side <code>CLIENT_EXIT_LOST_SERVER</code> notice from a helper/orphan client whose server was killed/aborted; benign	test isolation surface form, not a server crash	HIGH for "not a server crash" (70/70 clean); origin path = <code>client.c:210</code>

No product defect. The `prefix P` paste binding, `paste-buffer -p` into a login-shell pane, and the `tmux` server lifecycle are all sound.